

AGENTIC OPERATOR · 产品说明

Agentic Operator 产品 PRD(通俗版)

一群 AI 员工的指挥中心 —— 看得见、管得住、长得快

v0.2 · 大白话讲清楚 · 含 8 张图 · 2026-06-12



目录

1. 一、这是什么产品(一句话 + 一个比方)
2. 二、给谁用、帮他们解决什么头疼事
3. 三、我们的「AI 员工」(智能体)强在哪
4. 四、智能体舰队:一个面板管住一群 AI 员工
5. 五、会聊天的「追查助手」:Chatbot 怎么自己查、自己想
6. 六、三大本事:看得见 / 管得住 / 长得快
7. 七、它是怎么搭起来的(外行也能看懂)
8. 八、几样「看家硬功夫」
9. 九、现在做到哪、接下来做什么

一、这是什么产品(一句话 + 一个比方)

一句话说清

AO 是一群「AI 员工(智能体)」的指挥中心。这些 AI 员工帮你干活,比如筛简历、对岗位、查规则、发面试邀请;AO 让它们一起把活干完,同时让你能随时看清它们在干嘛、管得住它们、还能再添新人手。

现在 AO 主要带的是一支「招聘小队」——一群专门处理招聘的 AI 员工,从读简历、对岗位、查规则一直到约面试,一条龙接力。同一套指挥中心还被搬去带别的小队,比如管能源调度的、管费用的。

打个比方:像机场塔台

你可以把 AO 想成一座机场塔台。跑道上有很多架飞机(就是那些 AI 员工)在各自起飞、降落、忙活。塔台不替飞机开,但它把每架飞机看得清清楚楚:谁在飞、谁晚点了、谁出毛病了。塔台还能指挥:让某架先停一停、让某架重来一次。需要的时候,还能往机队里再添新飞机。AO 干的就是这件事——它不是来代替 AI 员工干活的,而是站在高处,把这群 AI 员工看清、管住、还能扩大。

它解决哪些头疼事

很多人以为,AI 员工最难的是「能不能干活」。其实真正头疼的是:它们上岗之后,没人看得清、管不住、也扩不动。AO 就是来收拾这几件麻烦的。

- 看不清:这群 AI 员工各干各的,活儿散落在各处。一出问题,你都搞不清到底是「它判断说这人不合适」(正常结果),还是「机器本身坏了」(故障)——这两件事很容易搞混,大半夜被惊醒还分不清该不该爬起来管。
- 管不住:AI 员工一上岗就像个黑盒子,你想让某个先停一停、改个设置、或者关键时候自己拍个板,都没有统一的地方去操作。
- 扩不动:每带一支新小队(比如从招聘换到能源调度),都得从零再搭一整套看板和规则,等于每次都重新造一遍轮子,太费劲。
- 靠不住:管这群 AI 员工要靠一个「运行引擎」,可它万一抖一下、歇一会儿,你的看板就跟着一起瞎了,啥都看不见了。外面合作的厂商(比如某个解析服务没钱了、或者 AI 接口坏了),它们的状态也没人统一盯着,经常被当成「AI 员工自己干砸了」错怪一通。

AO 把这四件头疼事拢到一个地方解决:让 AI 员工照常干活的同时,让管它们的人能一眼看清、随手管住、轻松扩大。哪怕那个运行引擎临时罢工,看板照样能从「数据库」里把历史记录翻出来给

你看,不会跟着一起瞎。

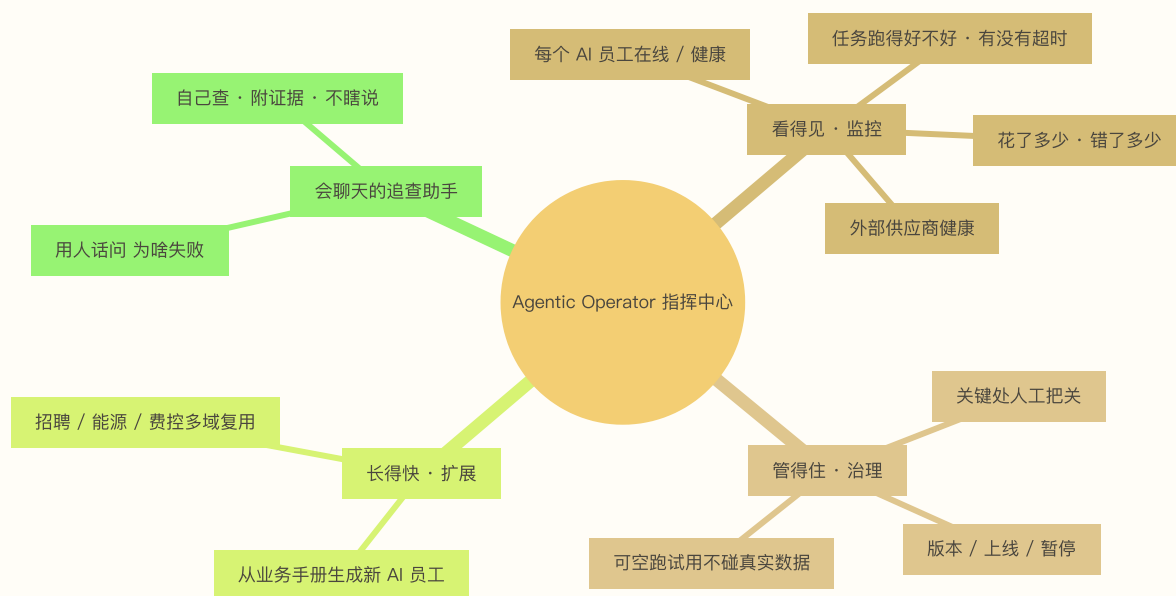
AO 想达到的三步走

- 先看得见:任何一个 AI 员工此刻在干嘛、跑得顺不顺、卡在哪、给出的结论靠不靠谱,你一眼就能看到;就算运行引擎临时出问题,也照样看得见。
- 再管得住:从「只能干看着」变成「能上手管」——能改设置、能喊停、关键时刻人能自己拍板,让 AI 员工从黑盒子变成你拿捏得住的对象。
- 最后长得快:想加个新 AI 员工、或者带一支全新的小队,不用从零搭,照着一套现成的「业务规则蓝图」就能快速生出来、直接上岗,让同一套指挥中心轻松复用到招聘以外的更多场景。

现状

成熟度(老实说):后端是真能跑的,不是花架子——招聘小队里的那些 AI 员工、查规则、关键时刻让人拍板、盯外面厂商的状态、给 AI 的结论做事实核对,这些都已经能用了。不过有些界面还停在「样子货」阶段:画面上的看板、卡片、还有员工名字客户名这类文字,有一部分是先摆出来的假数据占位。「管得住」里的动手操作(改设置、喊停这些)和「长得快」里加新员工的部分,还在做的路上,界面上很多地方是「看得见但暂时点不动」。带新小队这件事,目前招聘这一支最成熟,能源、费控等还在跟进。读后面章节时,以每节标注的「已经能用 / 还在做 / 计划中」为准。

图:产品功能地图 · 一眼看全貌



AO 把一群 AI 员工的「看得见 / 管得住 / 长得快」收进一个指挥中心,还配了会聊天的追查助手。

二、给谁用、帮他们解决什么头疼事

先说清楚一件事:这套指挥中心,不是给招聘里的人事或者求职者用的。它管的是一群干活的「AI 员工(智能体)」——比如帮你看简历、做匹配、查规则的那些。所以用它的人,是负责「盯着这群 AI 员工、别让它们出乱子、还要让它们越用越好」的那拨人。

打个比方:AI 员工像是一支夜班车间的机器人队伍,自己埋头干活。这套指挥中心就是车间里的中控室——大屏幕能看到每台机器在不在转、有没有卡壳,出了问题能一眼看出是机器坏了还是活儿本来就该这么干,还能随时按暂停、换班、加新机器。

2.1 三类人在用它

现在主要有三种人会用到这套系统。他们想要的东西不太一样,但有个共同心愿:把这群「时灵时不灵的 AI 员工」当成一套正经的生产设备来管,看得见、管得住、还能扩大规模。下面这张表把三类人、各自最想要什么、最头疼什么,摆在一起对照看。

用它的人	最想做到的事	最头疼的麻烦(这套系统就是来治这个的)
看摊子的运维 (管 AI 员工日常运转的人)	保证这群 AI 员工正常运转:谁停了、谁慢了、谁连不上外面的服务,第一时间发现,还能快速找出到底哪儿出了毛病	半夜一报警就分不清——到底是机器坏了该爬起来救火,还是活儿本来就该是这个结果根本不用管。出错的记录里,「业务结论」和「系统故障」混成一锅粥,看不出该不该叫人起床
盯结果的业务运营(看 AI 干得对不对、该不该让人拍板的人)	确保 AI 员工的判断是靠谱的(比如某个候选人到底合不合规、是过、是不过、还是要人再看一眼),并且管好那些需要人来最终拍板的待办事项	AI「失败」经常被误读成「这个人没通过」。更坑的是:这两件事的表现是反的——业务上没通过反而一声不响,真出系统故障才会大声报错。一不小心就照着错结论做了决定
管平台的负责人(管版本、算账、对外开口子的人)	管好业务规则蓝图的版本、AI 员工的版本和花销,还要能把 AI 的本事安全地借给外部合作伙伴去试用、跑测试	缺一个「固定版本、有记录可查、又不碰到正式生产」的对外试用通道;花了多少钱、外面哪家供应商欠费快断了,这些风险东一个西一个,没法集中算账和盯着

现状

成熟度(实话实说):三类人最关心的「看清楚现在怎么样」这一块,后台已经能真用了。把 AI 本事借给外部伙伴试用那个通道,第一版也已经上线、自己测过没问题。但面向运营的「人工动手干预」(比如远程改设置、按暂停)有一部分还在做、有些界面还是摆样子的占位。所以别光看界面长得齐全就以为全都能用了——下面每个小故事后面都标了「已经能用 / 还在做 / 计划中」,以那个为准。

2.2 四个真实的小场景

下面四个小故事,都是系统里真有的本事。它们想说明的是:这套系统不光告诉你「跑没跑」,更要回答「这回出岔子,到底是系统的锅还是活儿本来的结果、该不该叫人、能不能放心借给外面用」。

故事一:半夜批量出错——是系统坏了,还是这本来就是结果?

某个 AI 员工夜里批量看简历。运维收到一条提示,大意是「有候选人没通过规则校验」。这时候最容易吓了一跳:是不是系统崩了?但这套系统的门道在于:在查规则这条线上,「候选人没通过」从来不会被当成报错——它是个正常结论,安安静静地记下来。只有真出系统故障(比如连不上 AI 大脑、查不到资料库、工具转圈出不来、超时)才会大声报错。换句话说,你在记录里看到的那些「失败的步骤」,绝大多数其实是系统打了个嗝,根本不是哪个候选人被刷掉了。

- 实际怎么做的:系统会把每一步的结果分成四类——「顺利完成」「先卡了一下、重试后又好了」「系统故障(不是候选人的结果)」「真·业务结论(确实是过或不过)」。AI 写的那份总结,就照这个分类老老实实讲,不夸大。
- 好处:运维一眼就能分清「只是系统抖了一下、自己又缓过来了」和「真出了大事、得人来管」,不会被假警报吵醒,也不会把系统故障错当成业务结论。
- 状态:【已经能用】这套四档分类。而且看监控时,系统优先从自己的数据库里读;就算运行引擎短暂罢工,监控页照样能看。

故事二:外部供应商欠费 / 出故障——这是供应商的事,不能算到 AI 头上

AI 干活时要用到外面的一些服务(比如简历处理服务、AI 大脑接口)。要是这些外部服务欠了费或者自己坏了,AI 表面上看就是「干失败了」,但真正的原因压根不在自家这边。这套系统会在自己这边把供应商返回的信息读懂、归个类,把它单独记成「外部供应商的健康状况」,而不是混在一堆普通的「任务失败」里被淹没掉。

- 实际怎么做的:系统会按供应商分别汇总——最近一段时间里,哪家服务降级了、哪家正在报警(比如某家欠费了),口径跟报警中心是一致的。万一这个统计本身读不出来,它也只能说「暂时没拿全数据」,绝不会把整个总览页搞崩。

- 好处:平台负责人和运维能直接看到「是哪一家、从什么时候开始出问题、影响了哪些活儿」,把外面欠费/出故障和 AI 自己的毛病分开,各算各的账。
- 状态:【已经能用】供应商健康卡片 + 报警联动。而且对于「还能救回来」的故障,系统会先把那一摊活儿挂起、等会儿自动接着跑;只有真救不回来,才会标成彻底失败。

故事三:把 AI 借给外部伙伴去跑测试——用固定版本、不碰正式生产、还能留底备查

外部合作伙伴想借走那个「查规则的 AI 员工」,拿它跑一批回归测试(就是反复用同一批题,验证结果稳不稳)。这套系统专门开了一条独立通道:它完全绕开正式生产那条线——直接调用纯计算的部分,不发任何消息、不往正式账本里写东西、不会在外面产生任何副作用。所以伙伴在那边怎么折腾,都不会惊动正在跑的真实招聘业务。

- 实际怎么做的:伙伴提交一批题,系统先收下(同一个请求重复提交也只算一次,不会跑两遍),在后台跑,伙伴隔一会儿来问一次结果。进出都要凭专用钥匙(只对这条通道有效)。用的业务规则蓝图版本是写死的,要是报了个不认识的版本,系统直接拒绝、绝不偷偷换成别的。同时来的请求太多,系统会回一句「先稍等再来」,不会被冲垮。
- 实际怎么做的:这条通道只往自己专用的一张表里写东西,绝不碰正式账本;每跑一次都留一条独立的记录可查,不会弄脏正式招聘那边的画面。
- 好处:伙伴能在完全不影响线上的前提下,用固定版本对同一批题反复跑、结果可复现,还能把逐条规则的判定、用了哪些规则、花了多少用量都取回去做对账。
- 状态:【已经能用·第一版已上线并自测通过】支持单条简历匹配这个场景,目前是「只看不真改」的影子模式。⚠️【还在做】判定理由的细分拆解、以及把简历直接塞进请求里、还有计费口径,这几样还在对齐当中。

故事四:平时盯盘——健康、有没有超时、花了多少、谁要人拍板,一处看全

平时干活,运维和业务运营想要一块统一的大屏,把所有 AI 员工的状态盘一盘:都还健康吗?有没有谁干太久超时了?这阵子花了多少?有哪些事在等着人来拍板?还能按不同业务(招聘、能源、费控)分开看。这套系统的监控底盘是单独运转的,不跟被它盯着的那群 AI 挤在一块儿——这样万一 AI 那边出事,看盘的这块自己不会跟着一起趴窝。

- 实际怎么做的:监控底盘是一组独立运转的小程序,专门算健康、超时、花费、出错率、外部依赖这些指标,定时扫一遍,有问题就发提醒、记下来,不用改动任何现有数据。监控页就算碰上运行引擎出故障,也照样能看。
- 好处:把「跑没跑、有没有超时、花了多少、谁要人来管」收进一个统一口径里;提醒还能按业务分开看(目前能源、费控这些是真按业务分的,其余暂时还没分)。

- 状态:【已经能用】监控底盘 + 供应商健康 + AI 自动总结。【还在做 / 计划中】用 AI 多方交叉评审来抽查质量、以及「AI 表现有没有跑偏」的监测,还有一部分监控界面,都还在推进。

提示

这四个小故事说的其实是同一件事:面对「时灵时不灵的 AI 员工」,把「现在到底怎么样」「这回出岔子是系统的锅还是活儿的结果、是自己缓过来了还是真彻底完了、是自家问题还是外面供应商的问题」「用哪个版本、能不能放心借给外面用」,统统收进一个中控室里——让看摊子的、盯结果的、管平台的三种人,各取所需,把这群 AI 当成一套正经的生产设备来好好经营。

三、我们的「AI 员工」(智能体)强在哪

现在到处都能「随便接个 AI」让它帮你写两句话、答几个问题。但那种用法,像是雇了一个聪明却毛躁的临时工:它会瞎说、会中途撂挑子、出了岔子也没人知道,更没法放心交给它去碰公司真正的客户数据和钱。

AO 里的「AI 员工」不一样。我们不是简单地接了个 AI,而是在它外面包了一整套「靠谱上班」的规矩:干活分步骤、出事能补救、关键时刻喊人、说完话要被人核对、还会看自己用的外部工具有没有罢工。下面这份「优点清单」,每条都是一句话说优点、一句话说为什么好。

优点清单

- ① 干活分成一小步一小步,中途崩了能从断的地方接着干,不用从头再来。 — 就像搬家分箱打包,哪箱漏了只补那一箱;断电重启后接着搬,不会把已经搬好的又倒回去重搬一遍。
- ① 它还能「等人」:卡在某一步等你拍板时,会原地挂起、不占资源,你哪天回来批了它再继续。 — 等一整天、等一整夜都行,期间机器不空转、不超时报废,你的待办不会因为「等太久」就自己作废。
- ② 关键决定一定要人点头才放行,AI 只负责把活儿铺好。 — 比如调度放行、费用核准这种「拍错了会出大事」的环节,AI 把方案准备好、把利弊摆出来,真正按下「同意/退回/否决」的永远是人;责任和分寸攥在人手里。
- ③ AI 给的结论,会被另一套 AI 当「质检员」抽查,看它是不是有理有据、有没有瞎编。 — 一个人自说自话容易飘,我们让不同的 AI 互相对一下;意见不一致时还会多请几位当「评审团」投票,发现没依据的结论就标记出来报给人看,不让它蒙混过关。

- ④ 它分得清「是我手里的工具坏了」还是「这事按规矩本来就该不通过」。 — 这两件事完全不一样:工具坏了要修、要重试;按规矩没通过是正常业务结果,不该被当成故障乱报警。分清楚了,你才不会被一堆假警报淹没,也不会把真问题当成正常。
- ⑤ 它会盯着自己用的外部供应商,发现对方「没钱了 / 出故障了 / 说不准」,主动报给你。 — 它干活要用到外面的付费服务(比如简历解析、AI 大脑)。这些服务一旦欠费停了或宕机,它能从一次次失败里看出门道:是账户余额没了、还是对方服务器挂了、还是看不太准。它会归个类、提个醒,余额补上后受影响的活儿还会自动接着跑。
- ⑥ 想试试某个 AI 员工靠不靠谱?可以「空跑」一遍,只算不存,绝不碰真实数据。 — 像新员工先在模拟环境里走流程考核:全程纯演练,真实的客户档案、订单一个都不动,你看完它的表现满意了,再决定要不要让它上岗干真活。
- ⑦ 同一套「靠谱上班」的本事,招聘能用,换到能源调度、费用管控也照样能用。 — 上面这些能等人、能核查、能分辨故障、能空跑的真功夫,不是只为招聘写死的;换个行业、换套业务规则,这套底子直接搬过去就能用,不用从零再造一遍。

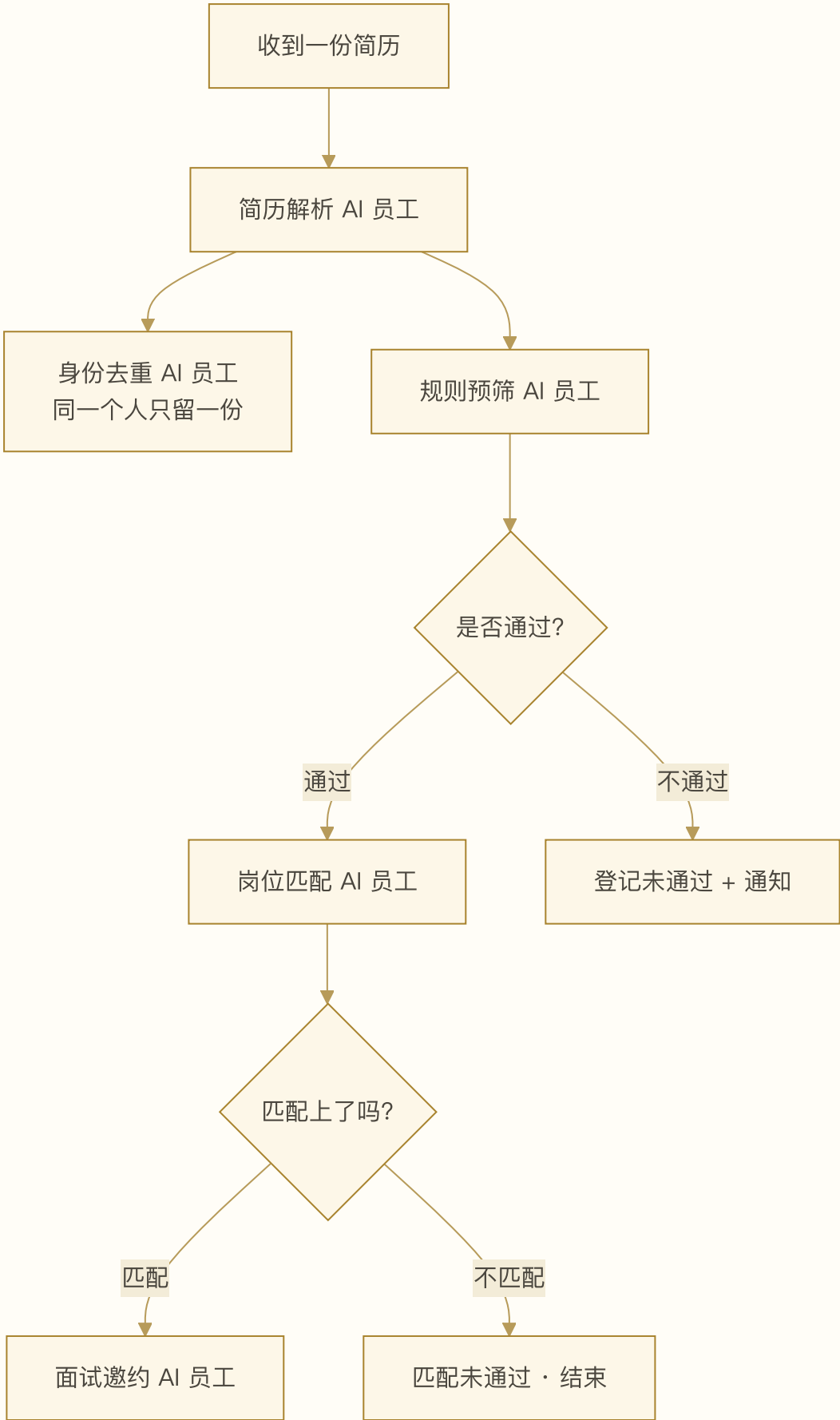
优点 × 它带来的好处(一张表看明白)

这个本事	对你来说意味着什么
分步干活、崩了能续、还能等人	断网、重启、宕机都不怕,活儿不会半路丢;该等人就稳稳等着,不空耗也不作废
关键处必须人点头	出大事的决定永远握在人手里,AI 只打下手,放心
产出被另一套 AI 抽查核对	AI 少瞎说,没依据的结论会被揪出来给你看,结果更可信
分得清「工具坏了」还是「正常没通过」	假警报少了,真问题不会被埋没,你不会被噪音淹
盯外部供应商没钱/故障	供应商一停摆你马上知道原因,补上之后活儿自动续跑,不用守着
可空跑试用、不碰真实数据	新 AI 员工先「试岗」再上岗,零风险考核,看顺眼了再用
一套能力跨行业复用	招聘之外(能源、费控)能快速复制,投入不白花,扩展快

现状

成熟度(说人话):前五条已经能用——招聘这条线的 AI 员工(简历解析、规则校验、面试邀约等)都在真跑;分步续跑、关键处人工把关、结论事实核查、分辨故障 vs 正常、感知供应商没钱/故障,这些都已落地在用。第六条「空跑试用」已经能用,目前主要给规则校验这类活儿做演练。第七条「跨行业复用」:招聘是真在跑的;能源调度、费用管控这两条线的底子已经搭好、也能跑通,但还在打磨,属于「还在做」。整体方向稳,部分能力还在持续完善。

图:AI 员工怎么接力干活(以招聘为例)



四、智能体舰队:一个面板管住一群 AI 员工

你雇了一群 AI 员工(我们叫它们"智能体")。有的专门看简历,有的负责做岗位描述,有的去匹配人岗,还有别的业务线(比如电力调度、费用管控)也各有一队。问题来了:它们平时各干各的、分散运行,你怎么知道谁在上班、谁在偷懒、谁出毛病了?"智能体舰队"这个页面,就是把这一群分散的 AI 员工,全部拉到同一块大屏幕上,像一个值班室的总控台,一眼看全、随手就能管。

打个比方:这就像一家公司的"员工花名册 + 考勤墙"。每个 AI 员工一行,名字、岗位、是在岗还是请假、最近干了多少活、活干得好不好、干一趟要多久——都明明白白列在那。它们还按"业务部门"分组(招聘部、能源部、费控部.....),你想看哪个部门,顶上一切换就只看那一队,不会一锅乱炖。

一眼能看到什么

- 谁在岗、谁请假、谁还没招进来:每个 AI 员工前面有个小圆点。绿色=在岗干活,灰色=被人手动暂停了(请假),红色=这个岗位只是规划里有、人还没真正建好(我们叫"蓝图")。
- 最近忙不忙、活干得好不好:每行直接显示"最近做了多少趟任务""成功率多少"(还配一根进度条,达标绿色、勉强黄色、不及格红色)、"干一趟通常要多久",以及"最后一次上工是几分钟/几小时前"。
- 谁需要你操心:顶上有几个分类标签——全部、在岗、需关注、蓝图。"需关注"会把那些在岗但最近成功率掉下来的 AI 员工单独挑出来,数字标红,值班的人扫一眼就知道今天该盯谁。
- 外部供应商有没有掉链子:页面最上方有一张"外部依赖健康卡"。AI 员工干活要用到外面花钱买的服务(比如简历解析、AI 大模型)。这张卡会告诉你这些服务是好的、还是"没钱了"、"故障了"、"说不准",哪些业务受影响,并直接给一个"清除"按钮让你处理。这样供应商一出事,你第一时间就看见,而不是等 AI 员工集体罢工才发现。

整块花名册大约每 5 秒自动刷新一次,所以数字是"活的",跟着实际情况走。如果后台某个数据源临时连不上,页面会老老实实标一句"部分数据缺失",不会假装一切正常。

点进单个 AI 员工,能看什么、能干什么

在花名册里点任意一行,右边会滑出一张"个人卡片";想看更全,还能进到这个 AI 员工的"个人主页"。在那里你能做这几件事:

- 看它的工作总览:最近一共做了多少趟、成几趟、败几趟、成功率多少、此刻有没有正在进行的任务,以及最后一趟的结果和耗时。想看每一趟的细节,一键就能跳到"运行监控"页深看。
- 看它最近经手了哪些对象:个人卡片会列出最近处理的几趟任务,而且写的是大白话——比如经手了哪位候选人、对应哪个岗位,而不是一串看不懂的编号。你能直观知道"它今天到底在忙谁的活"。
- 看它的版本档案,并给它换版本/上新版:每个 AI 员工的"做事配置"可以像存档一样保存成一个个版本。需要时,把某个旧版本一键"上线",它立刻就按那个版本干活;想退回上一版也是同样的操作。这相当于给每个 AI 员工配了"撤销键"和"版本回滚"。
- 把它上岗或请假:每行右边、以及个人主页上,都有一个"上线 / 下线"按钮。点一下就能让某个 AI 员工立刻开始干活,或者立刻停下,改动是即时生效的。
- 随时找它问问题:页面右下角有个常驻的"AI 助手"对话气泡。它知道你当前正看着哪个页面、哪个对象,可以直接用大白话问它"这个 AI 员工最近为什么失败多了"之类的问题,不用自己去翻一堆编号。

舰队能看什么 / 能管什么

你想做的事	在这个页面怎么做
看清谁在岗、谁请假、谁还没建好	花名册每行一个彩色小圆点:绿=在岗、灰=请假、红=只是蓝图还没建
按业务部门分着看	顶上切换部门(招聘 / 能源 / 费控.....),只看那一队
挑出今天要操心的	点"需关注"标签,自动把成功率掉下来的 AI 员工筛出来
看忙不忙、好不好、快不快	每行直接显示:最近任务趟数、成功率进度条、单趟耗时、最后上工时间
看外部供应商有没有出事	顶部"外部依赖健康卡":好 / 没钱了 / 故障了 / 说不准,并标出受影响的业务
看某个 AI 员工最近经手了谁	点进它,看最近处理的候选人 / 岗位(都是大白话,不是编号)
让它上岗或请假	点每行右边的"上线 / 下线"按钮,即时生效
给它换版本 / 退回上一版	进"版本档案",把目标版本一键上线;回滚同理
有疑问随时问	右下角"AI 助手"气泡,用大白话提问

哪些已经能用、哪些还在做

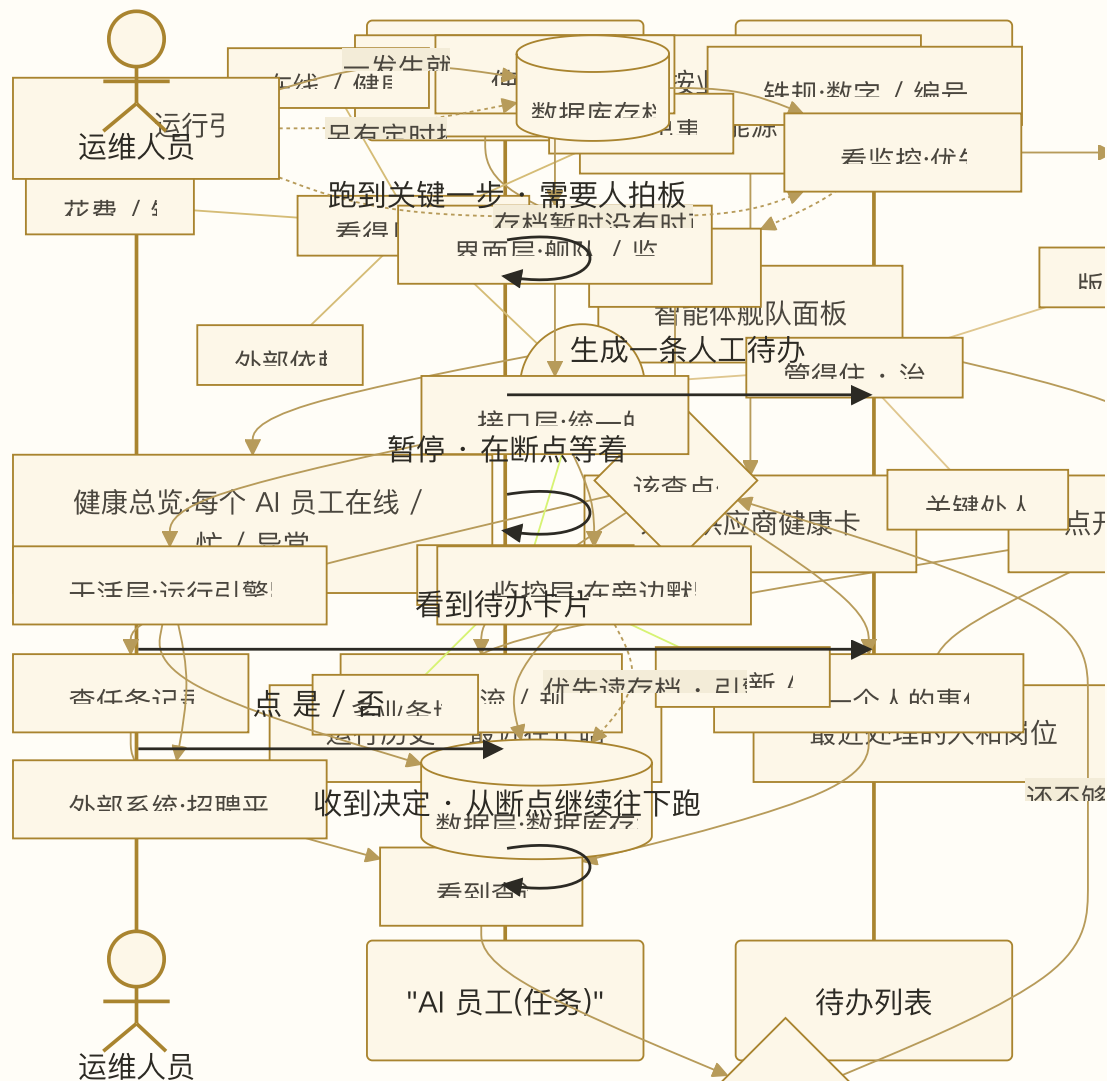
现状

已经能用:整块花名册(按部门分组、状态小圆点、最近趟数、成功率、耗时、最后上工时间)、需关注筛选、外部依赖健康卡、点进去的个人卡片和工作总览、最近经手对象的大白话展示、上线/下线开关、版本档案的保存与一键上线/回滚、右下角 AI 助手对话。

现状

还在做 / 有边界:① 真正完整在岗、能干活的 AI 员工目前是少数几个,大量是"暂停中的预置岗"或"只画了蓝图还没建"的占位;② 个人主页里的"运行明细 / 告警 / 消息 / 审计"几个标签页目前还是占位空页,真正深看要跳去"运行监控";③ "治理"下拉里的回滚、改归属、停用等动作目前是灰的(看得见、还点不动),等后续"管理"能力补齐;④ "换版本上线"现在只换"做事配置",不改底层代码——也就是能调它的工作方式,但深层改造仍需研发参与。

图:智能体舰队 · 一个面板管住一群 AI 员工



把分散运行的 AI 员工按业务域集中到一个面板:总览健康,点开单个还能看历史、查版本、上线、直接对话。

五、会聊天的「追查助手」:Chatbot 怎么自己查、自己想

AO 里住着一个会聊天的「追查助手」。你不用懂任何技术,直接用大白话问它就行,比如「上次那次任务为啥失败了?」「这个候选人到底走了哪几个环节?」它就像一个随叫随到的侦探,帮你把事情查个水落石出。

最关键的一点:它不是凭脑子里的印象瞎答。很多人担心 AI「张口就来、一本正经地胡说」,这个助手专门治这个毛病——它答任何一句话之前,都得先去翻真实记录。

它是怎么「自己查、自己想」的

打个比方:它手里有一串钥匙,一共 7 把,每把只能打开一个「档案柜」,而且只能看、不能改。这 7 个档案柜分别装着:所有任务的总台账、某一次任务的详细经过、系统里来来往往的消息流、规则审核的判定记录、候选人和岗位这些业务对象的信息、专门关失败任务的「故障收容箱」,还有一把特别的钥匙——能把同一个人、同一个岗位身上发生的事按时间先后串成一条「前因后果链」。

它不是一口气查完就完事,而是边查边想,像剥洋葱一样一层层来:先查一下,看到结果再决定下一步该往哪查,最多来回查 4 轮。比如你问「这次任务为啥失败」,它可能先翻总台账找到这次任务,看到有一步出错了,再去翻那一步的详细记录,最后才告诉你到底卡在哪。

更贴心的是,这整个「找线索」的过程你是看得见的。屏幕上会实时滚动告诉你它在干嘛:「正在查任务记录 → 查到 3 条 → 再去查这条的明细……」就像看侦探当着你的面办案,而不是关在小黑屋里给你一个结果。查完之后,它给出结论,还会在旁边附上「证据出处」的链接,你点一下就能跳到原始记录自己核对。

它还知道你「眼下正在看哪个页面、哪条任务」。所以你不用每次都把编号报一遍,直接问「这个为啥失败」,它就明白你说的是当前这条,答得更对路。

它能回答哪类问题

- 某次任务为什么失败了、卡在了哪一步
- 某个候选人从投简历到现在,一路都经历了哪些环节
- 最近这一天 / 这一周有哪些任务跑完了、哪些出了问题
- 某次规则审核为什么判了「不通过」,具体是哪条规则没过
- 某个岗位、某个客户名下都发生过哪些事,按时间顺序理一遍
- 现在「故障收容箱」里堆了哪些没跑成功的任务
- 某个编号(任务、候选人、审核记录)到底存不存在、是什么状态

如果你想让它「重跑一次」「取消」或者「改点东西」,它会礼貌拒绝,并提示你去专门的管理入口动手——它只负责看和说,绝不替你动手改。

不止一个:还有「专属版」

除了这个能查全局的「大管家」,还有两个同款的小助手:一个专门陪着「某一个 AI 员工」,你想盯着哪个智能体就问它的近况;另一个专门陪着「某一次任务」,你点开一次任务就能就地追问细节。本事都一样,只是看的范围更聚焦。

提示

差异化价值,就两条硬规矩:① 不编造——凡是涉及具体编号、数字、时间、状态的,统统必须来自真实查询,一个字都不许瞎编,从根上治住 AI「一本正经胡说」的老毛病;② 推理过程看得见——它查了什么、查到几条、下一步又去查什么,全程在你眼皮底下实时展示,最后还附上证据链接让你能亲自核对。这让它的回答既可信、又可追溯。

现状

成熟度:已经能用。聊天问答、自己分多轮查、实时展示查找过程、附证据链接、识别你当前所看页面,这些都已落地;三种助手(全局 / 单个 AI 员工 / 单次任务)也都已具备。前提是后台的 AI 服务已正常接通,否则它会提示你联系管理员。

图:追查助手怎么「边查边想」

它像侦探:自己调查询工具、看结果再决定下一步,最多查几轮,最后给结论加证据链接——全程看得见,且绝不编数字。

六、三大本事:看得见 / 管得住 / 长得快

前面说过,AO 是一群 AI 员工(智能体)的指挥中心。它的本事可以拆成三件事来理解:第一,把这些 AI 员工干活的情况看得清清楚楚(看得见);第二,人能随时对它们下指令、踩刹车、做调整(管得住);第三,需要新的 AI 员工时,不用从头搭,能快速长出来(长得快)。

打个比方:一家工厂里有一群机器人在流水线上干活。你得先有一面监控大屏,知道每台机器是不是在转、有没有卡壳、活儿干得对不对(看得见);你还得有一排开关和旋钮,能随时关停某台、调一调它的参数,遇到拿不准的活儿能叫人来拍板(管得住);最后,想多开一条新流水线时,得有一套现成的图纸和模具,照着就能装出新机器(长得快)。AO 就是这三样东西合在一起的指挥中心。

这三件本事是各管各、独立往前走的,谁也不耽误谁。其中“看得见”做得最齐全、天天在用;“管得住”和“长得快”都已经有了真东西能用了,但还没完全做完。下面一件一件说。

看得见(监控):我的 AI 员工现在怎么样?

这是三件本事里最成熟、用得最多的一件。它回答的就是一句最朴素的问题:我手下这些 AI 员工,现在还在干活吗?干得顺不顺?卡在哪了?就像车间主任抬头看一眼监控大屏,所有机器的状态一目了然。

- 一面统一的大屏(叫“智能体舰队”页):把每个 AI 员工的成功率、慢不慢、最近干了哪几单活,全都摆在一处,一眼看完。
- 几个专门的看板:谁干活慢了、有没有超时、花了多少用量、出错多不多,都有各自的板子盯着,自动报警。
- 盯外部伙伴的脸色:AI 员工干活要用到外面的服务(比如帮忙读简历的、出主意的大模型)。这些外部服务要是没钱了或者坏了,大屏顶上会专门亮一盏灯告诉你“是哪家、从什么时候开始出问题”,而不是把账算到自家 AI 员工头上。
- 断电也能看历史:就算后台那台“运行引擎”临时抽风、掉线了,大屏照样能从数据库里把过去的记录翻出来给你看,监控不会跟着一起“瞎掉”。这是 AO 特意做的一道保险。

现状

成熟度:已经能用(这一件做得最齐)。监控的主体骨架已经搭好并持续在用,断电兜底、外部伙伴健康灯都已经落地。少数更高级的玩法(比如让 AI 互相打分抽查、自动发现“跑偏”)还在做。

管得住(治理):我能对 AI 员工做哪些干预?

光看得见还不够,人得能动手。这一件回答的是:看到不对劲,我能做点什么?它就像监控大屏旁边那一排开关和旋钮——能关停某个 AI 员工、能调一调它的脾气(比如让它更稳重一点),遇到机器拿不准的活儿,还能由人来拍板。而且每一次人的操作都会被记成一笔账,事后能查谁动过什么。

- 踩刹车 / 松刹车:在大屏上直接一个开关,就能让某个 AI 员工停下来或者重新上岗。
- 调脾气:能改这个 AI 员工的一些设置,比如让它更谨慎、失败了多试几次。
- 人工把关:碰到 AI 拿不准的活儿,会变成一条“待办”交给给人,人点个“行”或“不行”,活儿再接着往下走。
- 记台账:人做的每一次干预都留痕,事后可查。

现状

成熟度:还在做(已经有真东西,但没完工)。关停/上岗、调设置、记台账这些是真能用的;但有些功能还是初版,而且现在还没接上“登录身份认证”——也就是暂时认不出具体是哪个人在操作。所以别把这一件当成“只是摆个样子”,它是“真的、但还没做全”。

长得快(扩展):我怎么长出新的 AI 员工 / 新的业务?

这一件回答的是:招聘这摊子用顺了,想再开一摊新业务(比如能源调度、费用管控),怎么不从零搭? AO 的办法是给一套现成的“业务规则蓝图生成器”——你把新业务的规矩讲清楚,它就能照着模具生成一套蓝图,而且生成出来的 AI 员工能直接跑起来。就像有了标准图纸和模具,装新机器、开新流水线就快得多。

- 业务规则蓝图生成器:给一个新业务“讲清规矩”,它就能生成一整套配套蓝图,而且能直接运行起来。招聘之外的能源调度、费用管控已经这么长出来了。
- 草稿到上岗的全流程:新 AI 员工可以先当“草稿”养着,再批量让它们上岗、下岗,过程不碰已有正式员工的活儿,互不打扰。
- 一个“部署新 AI 员工”的傻瓜式向导:这个还没做出来。现在点页面上那个“部署智能体”的按钮,打开的其实是“草稿库”,而不是一步步带你走完的完整向导;空白业务那里会提示“这个阶段请联系团队帮你部署”。

现状

成熟度:还在做。蓝图生成器、草稿养成与上岗下岗这些是真能用的;但面向普通人“点几下就装好一个新 AI 员工”的傻瓜式向导,还没交付,目前需要团队帮忙完成。

三件本事,一处看齐

这三件本事不是三个分开的网站,而是在同一个“智能体舰队”页上凑到了一块:在这一页里,你既能看见每个 AI 员工干得怎么样(看得见),又能就地按下关停/上岗的开关(管得住),还能从右上角进到“部署新员工”的入口(长得快)。它就是这三件本事的统一门面。

三大本事	回答的问题	能干什么(大白话)	做到什么程度
看得见(监控)	我的 AI 员工现在怎么样?	一面大屏看清谁在干、快不快、出没出错、花多少、外部伙伴是死是活;后台抽风了也能翻历史	已经能用(做得最齐)
管得住(治理)	我能做哪些干预?	关停/上岗、调脾气、人来拍板、人的操作记台账	还在做(真能用,但没做全、暂没登录认人)
长得快(扩展)	怎么长出新员工/新业务?	用蓝图生成器照模具长出新 AI 员工,新业务不用从头搭;草稿能养、能上下岗	还在做(傻瓜式向导未交付,需团队协助)

提示

一句话收尾:AO 想做的,就是让一群原本各跑各的 AI 员工,从“看得见”一步步走到“管得住”,再走到“长得快”——既能盯住它们、又能管住它们,还能随时再长出新的。

图:三大本事:看得见 / 管得住 / 长得快

监控这一轴已经能用,治理和扩展还在做 —— 三轴各自独立推进。

七、它是怎么搭起来的(外行也能看懂)

你可以把这套系统想成一家「全是 AI 员工的公司」。这家公司有前台、有干活的车间、有专门盯着大家的监控室、有放档案的仓库,还要和外面的合作方打交道。下面我们一层一层拆开看,保证不懂技术的人也能看明白。

现状 成熟度一句话:干活的车间、盯人的监控、放档案的仓库,这三块都「已经能用」;少数页面上的几张卡片还是「样板数据」(先把样子摆出来),还在补真实数据;部分对外的新功能「还在做」。下面每一层都会如实标。

整栋楼分五层

从上到下,这套系统大致分成五层。你可以照着「进门 → 找窗口办事 → 后台干活 + 旁边有人盯着 → 东西存进仓库 → 和外面对接」这个顺序来理解。

这一层叫什么	打个比方	干的活	成熟度
用户界面	公司前台和大屏幕	给人看的网页:谁在干活、干得怎么样、哪里出了问题,一眼看清	已经能用(少数卡片还是样板数据)
接口层	前台后面的办事窗口	把人点的每个按钮,翻译成系统能听懂的请求,再把结果端回来	已经能用
运行引擎(车间)	真正干活的车间	让 AI 员工一个接一个把活干完:写岗位、读简历、查规则、配人选	已经能用

这一层叫什么	打个比方	干的活	成熟度
监控(盯人的)	车间旁边的监控室	不进车间,在旁边盯着:谁卡住了、谁超时了、花了多少钱、哪里要叫人	已经能用
数据库(仓库)	放档案的大仓库	把干过的每件事、每条记录都存下来,以后随时能翻	已经能用
外部系统	楼外的合作伙伴	和外面的几家系统对接:取简历、用大模型、存文件、回写结果	已经能用

第一层:用户界面 —— 给人看的大屏

这是你打开网页第一眼看到的東西:菜单、看板、各种卡片。整栋楼的「门面」是统一的——左边的导航栏、顶上的标题栏,全部由同一套外壳负责,所以每个页面看起来风格一致,不会东一块西一块。每个页面只管摆自己的内容,外壳不用各画一遍。

界面支持中英文切换,也支持白天 / 夜间两种配色,点一下就整站换肤,不用重做。少数页面上的几张卡片现在还是「先摆个样子」的样板数据,真实数据还在往里接。

第二层:接口 —— 前台后面的办事窗口

你在网页上点一下,背后总得有人去办事。接口层就是这些「办事窗口」。它们按事情的类型分了好几个窗口:有专门管「看运行情况」的、有管「查规则」的、有管「人工拍板」的、有管「系统设置」的,还有专门给外面合作方用的窗口。人点按钮看数据走这里,外面合作方来取结果、回写信息也走这里。

提示

有个小讲究:监控相关的窗口去拿数据时,优先去「仓库」里翻档案,而不是直接冲进车间里抓。这一点很关键,下面专门讲。

第三层:运行引擎 —— 真正干活的车间

这是 AI 员工真正干活的地方,由一个「运行引擎」来调度。它的工作方式像一条传送带:一件事干完,自动触发下一件。比如招聘这条线就是——来了招人需求,先写岗位描述;简历下载好了,就去读简历、把信息整理出来;整理完,去查这个人符不符合规则;查通过了,再去做人岗匹配,最后发面试邀约。一个 AI 员工干完自己那段,自动把下一个叫起来,环环相扣。

目前真正在岗、干真活的 AI 员工有 6 个(写岗位、读简历、查规则、做匹配、发邀约,外加一个专门查「是不是同一个人、有没有重复」的)。另外还有一批「演示用的临时工」,平时关着,只在需要给大屏演示热闹效果时才打开,不会和真员工抢活。

不同业务是分开的车间。招聘是一个车间,能源调度是另一个独立车间,各管各的,互不串味。遇到需要人来拍板的环节(比如要不要放行),AI 员工会先停下来挂起,生成一条「待人处理」的事项;人点了「同意 / 不同意」之后,它再从原地接着往下干。这就是「人工把关」。

第四层:监控 —— 车间旁边的监控室(关键设计在这)

监控这套东西,故意不放在车间里跑,而是搭在旁边、单独一套。它是一组「老老实实算账」的小程序,定时去仓库里翻一遍记录,自己算出结论:谁卡住了、谁超时了、花了多少钱、出错多不多、哪里需要叫人。这样设计有个好处——盯人的人不和干活的人挤在一个屋里,车间再忙再乱,监控室自己不会被拖垮。

提示

为什么监控要单独搭?打个比方:你不会让仓库保安和搬货工人挤在同一个推车上。万一推车坏了,俩人一起趴窝——保安趴窝的那一刻,恰恰是最需要他报警的时候。所以监控必须独立。

重点讲清楚:为什么「看监控优先翻仓库档案」这么重要

这是整套系统最值得说的一个聪明设计,慢慢说。

干活的「运行引擎」本身,记着最新、最实时的现场情况——谁正在干、刚干到哪一步。它是现场的「真相」。问题是,这个引擎是个开发期用的临时角色,有时候会重启、会清空,偶尔「临时挂掉」。如果监控大屏直接盯着它看,它一挂,大屏立刻一片空白,你就成了「瞎子」——偏偏出事的时候最需要看清楚。

AO 的办法是:派一个「记录员」,在车间每发生一件事的当口,顺手抄一份存进仓库档案;同时还有一个「巡查员」,每隔半分钟左右再去车间扫一遍,把漏抄的、漏记的补齐。这样仓库里就攒下了一份完整的、长期保留的台账。

于是看监控时,系统优先去翻仓库里这份台账,而不是非得冲进车间盯现场。结果就是:就算运行引擎临时挂了,监控大屏照样能从仓库里读到过往的运行记录,不会跟着一起失明。这就是常说的「优先读数据库存档、运行引擎临时挂了也照样能看」。

看监控时怎么取数	说人话就是	什么时候用
自动(默认)	现场和仓库台账两边一起看,现场为准、仓库兜底,谁有就拼谁	平常态,既要最新、又要稳
只看仓库	完全只翻仓库台账,不碰现场	运行引擎挂了或离线时,照样能看

看监控时怎么取数	说人话就是	什么时候用
只看现场	只盯运行引擎的实时现场	调试、想看此刻最最新的状态

现状 成熟度:这套「记录员 + 巡查员 + 优先翻仓库」的容灾设计已经能用。需要诚实说明的小细节:巡查员补记会比现场最多晚半分钟左右,所以默认才用「两边一起看」,免得那些几十秒就跑完的短任务,在还没被抄进仓库前就被漏看。

第五层:数据库(仓库)与外部系统(楼外伙伴)

仓库就是数据库,放在本机上,实打实地存。干过的每一件事、每一条记录、每一份监控台账都存放在这里,以后随时能翻。AO 自己用的仓库和合作方那边的仓库特意分开放,各用各的门,互不打架。

楼外还有几家固定打交道的伙伴:一家负责帮忙读简历、生成岗位描述;一个是大模型(也就是 AI 的「大脑」),负责动脑子的活;一处专门存简历这类文件;还有一处存「业务规则蓝图」之类的关系数据;以及合作方那边的仓库,结果要回写给他们。这些对接都已经能用。

注意 一个实在的提醒:外面的伙伴会出状况——比如读简历那家「没钱了」,或者大模型「临时故障」。AO 早就把「外部伙伴不稳定」当成家常便饭来设计:它会在自己这边判断「这是伙伴的锅,不是我家 AI 员工的错」,把这种情况单独标成「依赖告警」,不会冤枉自己人。能自己缓过来的,先挂起等一会儿自动续上;实在缓不过来的,才正式记一次失败。

一张图理顺:东西是怎么流动的

- 干活的路:外面来了消息(比如新简历)→ 进车间 → AI 员工一个接一个干完 → 结果存进自家仓库、也回写给合作方,文件存好、关系数据存好。
- 存档的路:车间里发生的每件事 → 记录员当场抄一份、巡查员定时补齐 → 攒进仓库台账,长期保留。
- 看数据的路:你在网页上点开任意一页 → 经办事窗口 → 优先去翻仓库台账(现场挂了也能看)→ 监控结论由旁边那套独立小程序算好端上来。
- 对外的路:外面合作方想直接调一次「查规则」做测试 → 走专门的对外窗口 → 在一个不打扰生产的旁路里单独跑一遍,只写自己的小账本,绝不碰正式仓库,也不进主车间。

图:系统是怎么搭起来的(分层)

从上到下五层各司其职;关键设计:看监控优先读「数据库存档」,运行引擎临时挂了也照样能看。

八、几样「看家硬功夫」

前面讲了 AO 是「一群 AI 员工的指挥中心」。这一章不堆概念,只挑出 AO 真正下了功夫、别人不容易做到的五样本事,用大白话讲清楚:每样本事是为了堵住一个什么麻烦,堵住之后又给你带来什么好处。打个比方——它们就像一栋大楼里的几样隐形配置:停电了应急灯还亮、火警和有人做饭的油烟能分清、外面水电气哪根断了门口屏幕能显示、监控录像有人帮你复核真假、还允许外人在样板间里随便试用而不碰你家真东西。

现状

成熟度先说在前面(实话实说):下面第①第②第③样,已经能用、天天在跑;第④样(AI 帮你核对结论真假)主体流程已经能用,但「跟人工对答案校准准不准、模型有没有偷偷变笨」这两件配套的事还在做;第⑤样(对外开放试用)第一版已经上线、自己测过没问题,个别小功能还在补。

一、监控不会「跟着停电一起瞎」

麻烦在哪:AI 员工干活靠一个叫「运行引擎」的发动机带动,这台发动机本身是开发版的,偶尔会崩、会被清空。一旦它趴窝,如果监控大屏直接连着它,那大屏也跟着黑了——偏偏出事的时候你最需要看大屏。这就好比保安室的监控全靠主电,主电一断,整栋楼瞬间两眼一抹黑。

AO 怎么解决:它给监控加了一套「应急电+录像备份」。每当一次任务开始、结束或出问题的那一刻,AO 顺手就把这条记录复印一份存进自己的数据库;另外还有一个独立的小程序,每隔半分钟主动去把发动机里的最新情况抄一遍存下来,防止有漏抄。最后,所有大屏看数据时都默认先看「自己数据库里这份备份」,发动机还活着就拿它补最新的实时画面。三层叠在一起,等于监控有了自己的备用电源,跟发动机的死活解绑了。

- 带来的好处:发动机半夜崩了或被清空,你照样能看到今天跑过哪些任务、卡在了哪,不会两眼一抹黑。
- 一个贴心细节:有些任务几秒钟就跑完了,慢半拍的备份可能没来得及录到它「正在跑」那一下,所以 AO 选择把「实时画面」和「备份录像」叠加着看,而不是只看录像,免得漏掉这些一闪而过的快任务。

二、分得清「系统坏了」和「人没通过」,不把宕机说成淘汰

麻烦在哪:招聘场景里,AI 给一份简历判「不通过」是很正常的业务结论;但有时候 AI 根本没判,是因为它依赖的工具临时坏了(比如 AI 大模型没钱了、外部服务连不上、卡死打转)。这两种情况在原始记录里看起来都像「失败」,可意思天差地别——前者是「这个人确实没过」,后者是「机器抽风了,跟这个人没关系」。早期 AO 的自动小结就吃过这个亏:把一次系统抽风,写成了「候选人未通过」,看报告的人差点据此做了错误决定。这就像火警铃响了,你得先分清是真着火,还是只是有人在炒菜冒油烟。

AO 怎么解决:它定了一条铁规矩——真正的业务判断(通过/不通过)永远安安静静地写进结果里,不会「喊失败」;只有机器真的出毛病才会「喊失败」。于是 AO 把每一步自动归成四类:顺利完成、出过岔但重试后成功了、系统故障(跟人无关)、真实业务结论。自动小结严格照着这四类如实写,绝不把系统故障说成业务结论。

- 带来的好处:值班的人半夜被吵醒前,一眼就能分清「机器抽了一下、已经自己缓过来了」和「真出了需要人来管的事」,不再被假警报折腾,也不会把宕机当成淘汰结论去用。
- 更狠的一招:遇到这种「机器抽风」的故障,AO 不会硬把它写成「未通过」糊弄过去,而是把这次任务先挂起、重试,等工具恢复了再重新跑一遍,绝不让一次系统抖动冤枉了一个候选人。

三、一眼看清外面的「供应商」是死是活:没钱了 / 坏了 / 说不准

麻烦在哪:AI 员工干活要用到几个外部付费服务——解析简历的、提供 AI 大脑的等等。这些外面的「供应商」要是账户没钱了、或者服务器坏了,AI 这边就会成片地干活失败。可光看到「失败一大片」,你根本不知道到底是哪家、从什么时候开始、是没钱还是真坏了,只能干着急。这就像家里电器全不转,你却不知道是停电、欠费还是线路坏了。

AO 怎么解决:它专门盯着这些外部供应商返回的信号,按「哪家供应商 + 哪个业务」分门别类,给出三档明确结论,并在门口的健康卡片上显示出来:

结论	什么情况	建议怎么办
没钱了	明确收到了「额度用完」的信号,一次就能断定	去充值;充完之后,受影响的任务会自动接着跑

结论	什么情况	建议怎么办
坏了	全是干净的「连不上 / 超时」错误,而且不止偶尔一次	通常对方会自己恢复,等一等
说不准	信号里混进了模棱两可的东西(比如认证失败、限流)	AO 不装懂,先给个黄色提醒,持续没好转再升级,提示人工去核一下余额和服务状态

- 带来的好处:负责人能直接看到「是哪家、从什么时候起、影响了哪些活」,把「外面供应商的锅」和「我们自己 AI 的毛病」分清楚,不至于互相甩锅、查半天。
- 一个诚实的态度:AO 宁可承认「我说不准」,也不瞎猜。模棱两可时先黄灯、再升红灯,而不是一上来就乱报警吓人。

四、AI 给的结论,再请几个「不同门派的 AI」互相挑刺

麻烦在哪:前面几样管的是「机器有没有坏」,但还有个更要命的问题——AI 给出的结论本身对不对?它会不会一本正经地胡说,说一份简历匹配,其实根本没证据支撑?这种错误机器层面看不出来,因为任务「成功跑完」了,只是答案是错的。这就像监控录像一直在录,但录的内容到底是真是假,得有人复核。

AO 怎么解决:它会定期抽查最近的一批 AI 结论,请另一个 AI 来当「评审」,专门检查「这个结论有没有靠谱的证据撑着」。而且很讲究——这个评审 AI 一定要和原来出结论的 AI 是「不同门派」的,免得它护短自夸;遇到评审和原结论吵起来,就再叫上好几个互不相同的 AI 来投票,少数服从多数定个说法,整个投票过程都留底。

- 带来的好处:你不光知道 AI「跑没跑完」,还多了一层「它说的到底靠不靠谱」的体检,胡说八道更容易被逮住。
- 守住的底线:这层复核只看不改——它绝不会去篡改原来的业务结论,顶多发个提醒。万一评审 AI 自己也连不上,这次抽查就跳过,原结论照样算数,不会因为复核环节出问题反而把好结论搞乱。
- 成熟度:抽查、评审、投票这套主流程已经能用;但「定期跟人工标准答案对一对,看评审准不准」「盯着 AI 会不会偷偷变笨」这两件配套的事还在做。

五、对外开放「空跑试用」:外人能随便试,碰不到你的真东西

麻烦在哪:外部合作伙伴想拿 AO 的规则校验 AI 来做测试、跑回归,验证一批用例。但你总不能让外人直接在你正在生产、关系到真实候选人的流水线上乱跑——一旦串了线、写脏了真实数据,后果很严重。这就像别人想试你家厨房的灶,但你绝不能让他在你正在做饭的那口锅里折腾。

AO 怎么解决:它单独开了一个「样板间」式的试用通道,完全绕开真实生产流程——只做纯计算,不发任何消息、不写任何真实业务表、对外不产生任何副作用,对正常的招聘流水线零影响。试用产生的记录只写进一张独立的试用台账里,跟真实业务彻底隔开。

- 用的还是固定版本的「业务规则蓝图」:伙伴可以指定按哪一版规则来跑,版本对不上就直接报错、绝不偷偷换一版糊弄,保证同一批用例每次跑出来都能对得上、可复现。
- 有进门钥匙、也有限流闸:这个通道只认专属钥匙才放行,而且同时在跑的任务有上限,挤爆了会礼貌地让伙伴稍后再来,不会拖垮系统。
- 带来的好处:伙伴能放心大胆地在「样板间」里反复试、做体检,既拿得到逐条规则的结果和用量明细,又一根头发都碰不到你家的真实数据和线上流水线。
- 成熟度:第一版已经上线、自己测过没问题;目前支持「简历匹配」这一个场景,个别细节(比如按指定版本逐一固定执行、更复杂的输入方式)还在补。

提示

把这五样串起来看,AO 真正的看家本领是一句话:面对「AI 干活天生不那么确定」这件事,它没回避,而是反过来当成常态来设计——监控自己留后路、坏事先分清是机器还是业务、外面供应商的健康挑明了说、AI 结论再请人互查、对外开放也圈在样板间里。说白了,就是让你能放心地把这群 AI 员工当成一个正经的生产系统来管。

图:监控为什么「不失明」

事情一发生就顺手存进数据库,另有定时抄写兜底;看监控优先读存档,所以引擎重启也不丢、看板照样能看。

图:人工把关:暂停 → 等人点 → 继续

AI 员工跑到关键一步会暂停、生成一条待办,人点了「是 / 否」之后,它从断点接着往下跑。

九、现在做到哪、接下来做什么

这一章用大白话说清三件事:这个产品现在已经能用的有哪些、还在做的有哪些、以后才打算做的有哪些。我们不吹牛,把没做完的、有坑的也都老实摆出来。

打个比方:AO 就像一个工厂的总控制室,里面坐着一群 AI 员工(智能体)在干招聘、调度这些活。控制室要做到三件事——看得见这些 AI 员工在干嘛、管得住它们(能喊停、能让人插手)、还能再招新的 AI 员工进来。现在,看得见这一块基本盖好了,管得住和招新人这两块还在装修。

9.1 现在已经能用的

下面这些都不是花架子,是真能跑、真在用的:

- 看 AI 员工的状态:每个 AI 员工有没有在干活、有没有卡住、有没有超时、花了多少用量,都能在一个面板上看清。这套盯梢系统是单独运行的,不靠那群 AI 员工自己的运行引擎——好比保安和被保安看的人不是同一拨,这样就算运行引擎出毛病,你照样能看到现场。
- 断电也不瞎:AI 员工干的每一件事都会顺手存一份到数据库里。哪怕运行引擎临时崩了或者被清空,你回头还是能把之前发生的事一条条翻出来,不会两眼一抹黑。
- 事实核查:AI 员工做的判断(比如某份简历合不合规),会再请另一个不同牌子的 AI 来当裁判检查一遍,看它是不是有理有据、还是在一本正经地胡说。每次检查的结果、谁投了什么票、花了多少用量,都记下来存档。
- 分清是谁的锅:AI 员工『失败』了,到底是它自己不行,还是外面合作的工具(比如那个解析简历的服务没钱了、或者 AI 大脑临时连不上)的锅?系统会自己看一眼对方返回的信息,把外部故障单独

标成『依赖告警』,不让它混进 AI 员工自己的成绩单里。能自己恢复的就先挂起等会儿再续上,真修不好的才算彻底失败。

- 把脏活累活分四档:AI 员工干活留下的每一步,会被分成『成功 / 重试后成功 / 是外部设施坏了 / 是真做出了业务判断』四档。这样半夜报警时,值班的人一眼就知道该不该爬起来,不会被误报吵醒。
- 对外给合作伙伴试用的通道:外面的合作方想拿 AI 员工跑跑测试,可以走一条单独的小门。这条小门完全绕开正式生产线——不发消息、不动正式数据、不碰外面的东西,自己单独记一份账本。门口还有把锁(凭钥匙进),人太多了会礼貌地说『稍等会再来』。
- 人工把关:AI 员工干到关键路口会自己停下来,生成一张『请人拿主意』的待办;人点了是或否,它再从停的地方接着往下干。超时一直没人管,就自动作废并标记。

9.2 还在做的

这些已经动工、能跑一部分,但还没装修完,先别当成已经稳了:

- 让 AI 裁判更靠谱:现在 AI 当裁判核查事实、还能拉几个不同牌子的 AI 一起『陪审团』投票,这套已经接上了。但裁判判得准不准,还需要拿人工核对的结果去对照校准——这部分还没完全打通。
- 把裁判的判决都存进档案库:每次核查的判决会存进一张专门的台账,还能按不同业务分别设规矩(抽查多少、用哪个牌子的 AI 当裁判等)。这套在跑,但还在打磨。
- 对外试用通道的细节对齐:给合作伙伴的那条小门里,有些结果的说法(为什么这么判、证据是啥)目前是从一句话理由里临时拆出来的,还在和合作方对口径,包括怎么算用量。
- 档案保留多久还没定死:说好判决记录留 90 天,清理的活儿在干,但『到底留多久』这个时间还没正式设成可调的参数。

9.3 计划中(还没动或刚起步)

- 盯着裁判会不会『跑偏』:时间长了,AI 裁判的判断口味可能慢慢变。以后想定期拿老题目重判一遍,看它前后判得一不一致,偏得太多就报警。现在只有零件,还没串成自动流水线。
- 让人给裁判打分纠偏:以后想让人定期给裁判的判决打分,反过来纠正它。现在只能只读地看一张对错对照表,还没有让人录入打分的入口。
- 一个专门管监控、看『跑偏』的界面:目前还没有这个可视化界面。
- 让监控能自己动手干预:现在监控最多只会『通知你』,还不能自己出手去暂停或修。让它自己动手这一档,以后才做。

9.4 已知的坑和没做完的(老实交代)

注意 有些界面还是『样子货』:控制室的一些页面和里面的名字(比如 AI 员工的名字、客户名)目前是写死的假数据,后台是真的、但前台还没跟真数据接通。所以对外说『实时监控』时一定要分清楚:哪些卡片是真数据、哪些只是占位摆设,别把摆设当成真本事承诺出去。

注意 有些任务追不到完整的来龙去脉:事实核查那条旁路上,任务缺一个能串起来的『追踪编号』,很多编号是临时拼出来的假号。结果就是,想顺着一个人一路追到底、看完整链条时会断档。流程追踪页已经做了降级处理(追不到就少显示一点),但把这些编号补齐串通,还是个没解决的活儿,排在后面再做。

注意 事实核查管得还不够全:目前 AI 裁判只核查招聘这块的活儿。能源调度、费控那些业务虽然也进了基本监控,但它们的 AI 输出还没纳入这套事实核查,等于有一片没盯到的死角。

注意 和合作方『两边同时记账』还没谈拢:AO 和合作方 RAAS 之间,有些场景需要两边同时写数据、同时发消息。如果两边的先后顺序对不上,可能出现重复或漏发。这事得等双方把规矩正式定下来才能彻底固定。

注意 对外试用通道只有一把总钥匙:给合作伙伴的那条小门现在只有一把共用钥匙,没法给不同人发不同钥匙、也没法定期换锁;人太多时的限流也只是『软提醒』,极端情况下可能有点小抢位。还有,当实时源和备份档案同时都连不上时,系统会直接报错停下——这是有意为之的『宁可停也不出错』,但属于极少见的极端情况。